NARAMMALA CHANNELING HOSPITAL

Hospital Management System

MICROSERVICES API SPECIFICATION

Complete endpoint reference for solo developer implementation

Architecture	6 Microservices
Database	PostgreSQL 15 (schema-per-service)
Auth	Keycloak (realm: hms)
File Storage	MinIO (S3-compatible)
Messaging	RabbitMQ (async events)
Runtime	Node.js 20 + Express · Docker Compose

1. Global Conventions

All services follow these conventions. Read this once and apply to every service.

1.1 Base URL Pattern

Service Ports

Reception Service → http://localhost:3001
Clinical Service → http://localhost:3002
Staff Service → http://localhost:3003
Finance Service → http://localhost:3004
Lab Service → http://localhost:3005
Ward Service → http://localhost:3006
API Gateway → http://localhost:3000 (clients always use this)
Keycloak → http://localhost:8080
MinIO → http://localhost:9000
RabbitMQ Mgmt UI → http://localhost:15672

1.2 Authentication

Every request (except Keycloak's own login endpoint) must include a valid Keycloak JWT in the Authorization header. The API Gateway validates the token before forwarding to any service. Services themselves just trust the injected headers — they never call Keycloak directly for validation.

How auth works in practice

1. Client calls POST http://localhost:8080/realms/hms/protocol/openid-connect/token
   Body: { grant_type: password, client_id: hms-app, username, password }
   Returns: { access_token, refresh_token, expires_in: 900 }

2. Client includes header on every API call:
   Authorization: Bearer <access_token>

3. API Gateway extracts token, fetches Keycloak public key (JWKS), verifies signature
4. Gateway injects headers into upstream request:
   X-User-Id: <user uuid from Keycloak>
   X-User-Role: <role from token claims>
5. Service reads X-User-Id / X-User-Role — no auth code needed in services

1.3 Standard Response Format

Every API response follows this envelope:

```
// Success
{
  "success": true,
```

```
  "data": { ... },
  "meta": { "page": 1, "limit": 20, "total": 143 }  // only on paginated lists
}

// Error
{
  "success": false,
  "error": {
    "code": "PATIENT_NOT_FOUND",
    "message": "No patient with id abc-123",
    "statusCode": 404
  }
}
```

1.4 HTTP Status Codes

Code	Meaning	When to use
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST that creates a resource
204	No Content	Successful DELETE
400	Bad Request	Invalid input / validation error
401	Unauthorized	Missing or invalid JWT
403	Forbidden	Valid JWT but wrong role for this endpoint
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate (e.g. patient already registered with that mobile)
422	Unprocessable	Business rule violation (e.g. bed already occupied)
500	Server Error	Unexpected server error

1.5 File Upload Convention (MinIO)

When an endpoint accepts file uploads, use multipart/form-data. The service uploads the file to MinIO and stores only the object key in the database. To serve the file back to clients, the service generates a pre-signed URL (15-minute TTL).

File upload flow

POST /clinical/sessions/:id/images
Content-Type: multipart/form-data
Body: { image: <file>, caption: 'Left knee X-ray' }

Service: validates MIME (image/jpeg, image/png, application/pdf) → uploads to MinIO → stores key
Response: { file_url: 'https://minio:9000/hms-scans/clinical/sess-uuid/uuid.jpg?X-Amz-...' }
URL is pre-signed, expires in 15 minutes. Fetch a fresh URL via GET /sessions/:id if needed.

1.6 Pagination

All list endpoints support: ?page=1&limit=20&sort=created_at&order=desc

Response meta field: { page, limit, total, totalPages }

2. Reception Service · :3001

Purpose & Responsibility

Manages everything at the front desk: patient registration, queue token issuance, appointment booking.

This service owns the reception schema in PostgreSQL with these tables:
patients — all registered patient records
tokens — queue tokens issued per day per queue type
appointments — scheduled doctor appointments

Key behaviour: when a token status changes, it publishes a queue.updated event to RabbitMQ, which the API Gateway WebSocket bridge picks up and pushes to Queue Display boards in real time.

2.1 Patient APIs

Method	Endpoint	Role(s)	Description
POST	/patients	receptionist	Register a new patient. Creates record and returns patient_number (e.g. R00001).
GET	/patients	admin, receptionist	Paginated list of all patients. Supports ?search=name query.
GET	/patients/search	receptionist, doctor, nurse	Search patients by name or mobile. Returns lightweight list for autocomplete. ?q=john
GET	/patients/:id	receptionist, admin, doctor, nurse	Get full patient record by ID.
PUT	/patients/:id	receptionist	Update patient details (address, contact, etc).

POST /patients — Request Body

Field	Type	Null	Notes
first_name	string	No	Patient first name
last_name	string	No	Patient last name
mobile	string	No	Mobile number — used for search and SMS. Must be unique.
email	string	Yes	Optional email
date_of_birth	date	No	Format: YYYY-MM-DD
gender	enum	No	male | female | other
address	string	Yes	Full address text
blood_group	string	Yes	A+ A- B+ B- O+ O- AB+ AB-
triage	enum	Yes	non_urgent | emergency | out_patient | urgent
avatar	file	Yes	Multipart: patient photo — uploaded to MinIO hms-avatars/

Auto-generated on POST /patients

patient_number — sequential, human-readable: R00001, R00002…
id — UUID v4
created_at — server timestamp
status — defaults to 'active'

2.2 Token / Queue APIs

A token represents a patient's place in a queue for a given day. Three queue types exist: opd, wound_care, doctor_consultation. Token numbers reset daily.

Method	Endpoint	Role(s)	Description
POST	/tokens	receptionist	Issue a queue token for a patient. Returns token number and position.
GET	/queue/:type	receptionist, doctor, nurse	Get current live queue for a type. Returns all tokens sorted by token_number with status. Used by Queue Display board.
PUT	/tokens/:id/status	receptionist, doctor, nurse	Update token status. Publishes queue.updated event to RabbitMQ on every change.
GET	/tokens/today	receptionist	All tokens issued today across all queues.

POST /tokens — Request Body

Field	Type	Null	Notes
patient_id	UUID	No	Must be a registered patient
queue_type	enum	No	opd | wound_care | doctor_consultation
doctor_id	UUID	Yes	Required when queue_type = doctor_consultation

PUT /tokens/:id/status — Request Body

Field	Type	Null	Notes
status	enum	No	waiting | called | in_progress | completed | skipped | cancelled

RabbitMQ event published on every token status change

Exchange: queue.events (fanout)
Routing key: queue.updated
Payload: { token_id, token_number, patient_name, queue_type, status, doctor_id?, updated_at }

API Gateway subscribes to this exchange and broadcasts via Socket.IO to all connected Queue Display browser clients — no polling, instant update.

2.3 Appointment APIs

Appointments are for scheduled doctor consultations. Receptionist books these in advance. The appointment list is the core of the Admin and Doctor dashboards.

Method	Endpoint	Role(s)	Description
POST	/appointments	receptionist	Book a new appointment. Checks doctor schedule for slot availability before saving.
GET	/appointments	admin, receptionist, doctor, nurse	List appointments. Doctor sees only own. Supports filters: ?doctor_id=&date=&status=
GET	/appointments/:id	admin, receptionist, doctor, nurse	Single appointment detail.
PUT	/appointments/:id	receptionist	Edit appointment (reschedule, change doctor, update notes).
DELETE	/appointments/:id	receptionist, admin	Cancel appointment.

POST /appointments — Request Body

Field	Type	Null	Notes
patient_id	UUID	Yes	If null, patient_name/mobile are used (unregistered walk-in)
patient_name	string	Yes	Required if patient_id is null
patient_mobile	string	Yes	Required if patient_id is null
gender	enum	Yes	male | female | other
address	string	Yes	
doctor_id	UUID	No	Consulting doctor
appointment_date	date	No	YYYY-MM-DD
from_time	time	No	HH:MM (24h)
to_time	time	No	HH:MM (24h)
treatment	string	Yes	Reason / treatment type
notes	string	Yes	Additional notes
avatar	file	Yes	Appointment-related image — MinIO

3. Clinical Service · :3002

Purpose & Responsibility

Manages everything that happens during a patient's clinical visit: OPD consultations, wound care sessions, and specialist channelling. Doctors and nurses use this service to record diagnoses, prescriptions, vitals, upload scan images, and request lab tests.

Database: clinical schema in PostgreSQL (sessions, prescriptions, vitals, diagnoses tables)
File storage: MinIO (hms-scans/ for images)

Key events published:
billing.opd → on OPD session complete → Finance creates invoice
billing.wound → on wound care complete → Finance creates invoice
lab.requested → on lab test request → Lab Service creates order
pharmacy.new_rx → on internal Rx saved → WebSocket push to Pharmacy Display

3.1 Session APIs

A session represents one patient visit. Must be created first, then updated incrementally as the doctor records information.

Method	Endpoint	Role(s)	Description
POST	/sessions	doctor, nurse	Start a new clinical session for a patient. Returns session_id used for all subsequent calls.
GET	/sessions	doctor, nurse	List sessions. Doctor sees only own. Supports ?patient_id=&date=&type=
GET	/sessions/:id	doctor, nurse	Get full session including all sub-records (vitals, diagnoses, prescriptions).
PUT	/sessions/:id	doctor, nurse	Update session fields: chief_complaint, notes, diagnosis text, follow_up_date.
PUT	/sessions/:id/complete	doctor	Mark session as complete. This triggers the billing event to Finance.

POST /sessions — Request Body

Field	Type	Null	Notes
patient_id	UUID	No	
session_type	enum	No	opd | wound_care | channelling
appointment_id	UUID	Yes	Link to appointment if pre-booked
chief_complaint	string	Yes	Patient's main symptom description

PUT /sessions/:id/complete — Request Body

Field	Type	Null	Notes
diagnosis	string	No	Final diagnosis text
discount_percent	number	Yes	0-100. 0 = no discount
discount_reason	string	Yes	Required if discount_percent > 0

PUT /sessions/:id/complete — what happens internally

1. Status set to completed
2. Finance billing event published:
   Exchange: billing.events, routing key: billing.opd (or billing.wound)
   Payload: { session_id, patient_id, patient_name, session_type, doctor_id, consultation_fee, discount_percent, discount_reason, completed_at }
3. Finance Service receives event and creates invoice automatically

3.2 Vitals APIs

Method	Endpoint	Role(s)	Description
POST	/sessions/:id/vitals	doctor, nurse	Record patient vitals for this session.
GET	/sessions/:id/vitals	doctor, nurse	Get vitals recorded for this session.
GET	/patients/:id/vitals-history	doctor, nurse	All historical vitals for a patient. Used in Patient Dashboard health chart.

POST /sessions/:id/vitals — Request Body

Field	Type	Null	Notes
bpm	integer	Yes	Heart rate in beats per minute
temperature	decimal	Yes	Temperature in Celsius
blood_pressure_sys	integer	Yes	Systolic blood pressure mmHg
blood_pressure_dia	integer	Yes	Diastolic blood pressure mmHg
spo2	integer	Yes	Oxygen saturation %
weight_kg	decimal	Yes	Weight in kilograms
height_cm	integer	Yes	Height in centimetres
blood_sugar	decimal	Yes	Blood glucose mmol/L

3.3 Diagnosis & Image APIs

Method	Endpoint	Role(s)	Description
PUT	/sessions/:id/diagnosis	doctor	Save or update diagnosis text for the session.

POST	/sessions/:id/images	doctor, nurse	Upload diagnosis or wound image. Multipart upload → MinIO hms-scans/. Returns pre-signed URL.
GET	/sessions/:id/images	doctor, nurse	List all images for a session with pre-signed URLs.
DELETE	/sessions/:id/images/:imageId	doctor	Delete a session image.

POST /sessions/:id/images — Request Body (multipart/form-data)

Field	Type	Null	Notes
image	file	No	Image file. Accepted: image/jpeg, image/png. Max 10MB.
caption	string	Yes	Label for the image e.g. 'Right knee anterior view'
image_type	enum	Yes	scan | wound | other

3.4 Prescription APIs

Two types of prescriptions: internal (dispensed by hospital pharmacy) and external (patient takes to outside pharmacy). External prescriptions can be printed as PDF via the Finance Service.

Method	Endpoint	Role(s)	Description
POST	/sessions/:id/prescriptions	doctor, nurse	Add one or more prescriptions to the session.
GET	/sessions/:id/prescriptions	doctor, nurse	Get all prescriptions for a session.
DELETE	/sessions/:id/prescriptions/:rxId	doctor	Remove a prescription entry.
GET	/pharmacy/queue	pharmacist (via gateway)	All active internal prescriptions pending dispensing. Used by Pharmacy Display board.
PUT	/pharmacy/:rxId/dispense	pharmacist	Mark internal prescription as dispensed.

POST /sessions/:id/prescriptions — Request Body

Field	Type	Null	Notes
prescriptions	array	No	Array of prescription objects (see below)
prescriptions[].type	enum	No	internal | external
prescriptions[].medicine_name	string	No	
prescriptions[].dosage	string	No	e.g. '500mg'
prescriptions[].frequency	string	No	e.g. 'twice daily', 'after meals'
prescriptions[].duration_days	integer	No	Number of days

prescriptions[].instructions	string	Yes	Special instructions
prescriptions[].pharmacy_name	string	Yes	External prescriptions only — preferred pharmacy

3.5 Lab Request APIs

Doctors request lab tests from within the clinical session. This publishes a RabbitMQ event to the Lab Service which creates a pre-populated order.

Method	Endpoint	Role(s)	Description
POST	/sessions/:id/lab-requests	doctor	Request one or more lab tests. Publishes lab.requested event. Returns request_id.
GET	/sessions/:id/lab-requests	doctor, nurse	Get all lab test requests for this session with current order status.

POST /sessions/:id/lab-requests — Request Body

Field	Type	Null	Notes
tests	array	No	Array of { test_id, urgency: routine|urgent } objects
notes	string	Yes	Instructions for lab technician

3.6 Patient History API

Method	Endpoint	Role(s)	Description
GET	/patients/:id/history	doctor, nurse, receptionist	Full clinical history for a patient: all sessions, diagnoses, vitals trend, medication list.

4. Staff Service · :3003

Purpose & Responsibility

Manages doctors, departments, availability schedules, and nurse allocations.

Important: when a doctor is created (POST /doctors), this service calls the Keycloak Admin API to create the user account in the hms realm. When deleted, it deactivates the Keycloak account. Doctors can then log in with those credentials immediately.

Database: staff schema — doctors, staff, departments, schedules, nurse_allocations tables
File storage: MinIO hms-avatars/ and hms-doctor-banners/

4.1 Doctor APIs

Method	Endpoint	Role(s)	Description
GET	/doctors	admin	Paginated list of doctors with department and specialisation.
POST	/doctors	admin	Create doctor profile AND Keycloak user account simultaneously.
GET	/doctors/:id	admin, doctor	Doctor full profile including about, skills, education, experience.
PUT	/doctors/:id	admin	Edit doctor details.
DELETE	/doctors/:id	admin	Soft-delete (status = inactive) and deactivate Keycloak account.

POST /doctors — Request Body

Field	Type	Null	Notes
first_name	string	No	
last_name	string	No	
username	string	No	Keycloak username — also becomes their login
email	string	No	
mobile	string	No	
password	string	No	Initial password for Keycloak account
date_of_birth	date	No	YYYY-MM-DD
gender	enum	No	male | female | other
education	string	No	Qualifications e.g. MBBS, MS
designation	string	No	e.g. Senior Doctor, Physician
department_id	UUID	No	

address	string	Yes	
city	string	Yes	
country	string	Yes	
state_province	string	Yes	
postal_code	string	Yes	
biography	string	Yes	Doctor's short bio
avatar	file	Yes	Multipart: doctor photo → MinIO hms-avatars/
status	enum	No	active | inactive

What happens internally on POST /doctors

1. Validate all fields
2. Call Keycloak Admin API: POST /admin/realms/hms/users with { username, email, credentials, enabled: true }
3. Assign role 'doctor' in Keycloak
4. Save doctor profile to staff.doctors table with Keycloak user_id
5. Upload avatar to MinIO if provided
6. Return created doctor record

4.2 Department APIs

Method	Endpoint	Role(s)	Description
GET	/departments	any authenticated	List all departments.
POST	/departments	admin	Create a new department.
PUT	/departments/:id	admin	Update department.
DELETE	/departments/:id	admin	Delete department (only if no active doctors assigned).

4.3 Doctor Schedule APIs

Schedules define when a doctor is available for appointments. The Reception Service reads these to validate appointment slots.

Method	Endpoint	Role(s)	Description
GET	/doctors/:id/schedule	admin, doctor, receptionist	Get all schedule entries for a doctor.
POST	/doctors/:id/schedule	admin, doctor	Add a schedule slot (day of week + time range).
PUT	/schedules/:id	admin, doctor	Update a schedule entry.

DELETE	/schedules/:id	admin, doctor	Remove a schedule entry.
GET	/doctors/:id/availability?date=	receptionist	Get available appointment slots for a doctor on a given date. Returns free time slots.

POST /doctors/:id/schedule — Request Body

Field	Type	Null	Notes
day_of_week	integer	No	0=Sunday, 1=Monday … 6=Saturday
start_time	time	No	HH:MM
end_time	time	No	HH:MM
slot_duration_minutes	integer	No	Duration of each appointment slot e.g. 15
max_patients	integer	No	Maximum appointments in this session

4.4 Nurse Allocation APIs

A nurse must be allocated to a doctor's session before the session begins. The allocation expires when the session is marked complete. A nurse can only be allocated to one doctor at a time.

Method	Endpoint	Role(s)	Description
POST	/nurse-allocations	admin, receptionist	Allocate a nurse to a doctor for a specific session date. Checks nurse not already allocated.
GET	/nurse-allocations	admin	List allocations. Supports ?doctor_id=&date=&status=
PUT	/nurse-allocations/:id/complete	admin, nurse	Mark allocation as complete — frees nurse for next allocation.
DELETE	/nurse-allocations/:id	admin	Cancel a nurse allocation.
GET	/staff/available?date=&role=nurse	admin, receptionist	Get staff members available for allocation on a given date (not already allocated).

5. Finance Service · :3004

Purpose & Responsibility

Handles all billing across every module. Invoices are created in two ways:

1. Automatically — by consuming RabbitMQ events (billing.opd, billing.wound, billing.ward, billing.lab)
2. Manually — via POST /invoices for walk-in billing at reception counter

Database: finance schema — invoices, line_items, payments tables
File storage: MinIO hms-prescriptions/ for printed Rx PDFs

Finance Service never calls other services. It only listens to events and responds to HTTP.
The receptionist uses this service at the billing counter to take payments and print receipts.

5.1 Invoice APIs

Method	Endpoint	Role(s)	Description
GET	/invoices	admin, receptionist	Paginated invoice list. Supports ?module=&status=&date=&patient_id=
POST	/invoices	receptionist, admin	Manually create invoice (for walk-in billing). Event-driven invoices are auto-created.
GET	/invoices/:id	admin, receptionist	Full invoice detail with all line items.
PUT	/invoices/:id/pay	receptionist	Record payment. Updates payment_status and paid_at.
POST	/invoices/:id/discount	admin, doctor	Apply or update discount on an invoice.
GET	/invoices/:id/print	receptionist, admin	Returns invoice data structured for receipt printing.

POST /invoices — Request Body (manual billing)

Field	Type	Null	Notes
patient_id	UUID	No	
billing_module	enum	No	opd | wound_care | channelling | lab | ward | pharmacy
session_reference_id	UUID	Yes	ID of the clinical session or lab order
line_items	array	No	Array of { item_name, item_type, quantity, unit_price }
notes	string	Yes	

PUT /invoices/:id/pay — Request Body

Field	Type	Null	Notes
payment_method	enum	No	cash | card | online | insurance

amount_paid	decimal	No	Actual amount paid (may differ from total if partial)
notes	string	Yes	Payment notes

POST /invoices/:id/discount — Request Body

Field	Type	Null	Notes
discount_type	enum	No	percentage | flat
discount_value	decimal	No	Percentage (0-100) or flat amount
reason	string	No	Reason for discount — stored for audit

5.2 Report APIs

Method	Endpoint	Role(s)	Description
GET	/reports/earnings	admin	Earnings summary by period. ?period=monthly&year=2026. Returns monthly totals for chart.
GET	/reports/departments	admin	Patient count and revenue per department.
GET	/reports/summary	admin	Dashboard summary: total appointments, new patients, earnings, operations count + % vs last period.

5.3 Prescription Print API

Method	Endpoint	Role(s)	Description
POST	/prescriptions/print	receptionist	Generate external pharmacy prescription PDF from session prescriptions. Stored in MinIO, returns download URL.

POST /prescriptions/print — Request Body

Field	Type	Null	Notes
session_id	UUID	No	Clinical session containing the external prescriptions to print
doctor_name	string	No	Printed on the prescription
hospital_name	string	Yes	Defaults to 'Narammala Channeling Hospital'

5.4 RabbitMQ Event Consumers

Finance Service subscribes to these RabbitMQ events and auto-creates invoices

billing.opd payload: { session_id, patient_id, patient_name, doctor_id, consultation_fee, discount_percent, completed_at }
→ Creates invoice with line_item: { item_name: "Doctor Consultation", unit_price: consultation_fee }

billing.wound payload: { session_id, patient_id, patient_name, wound_care_fee, discount_percent, completed_at }
→ Creates invoice with wound care fee line item

billing.ward payload: { admission_id, patient_id, patient_name, services[], total_amount, discharged_at }
→ Creates invoice with all ward service line items

billing.lab payload: { order_id, patient_id, patient_name, tests[], total_amount, paid_at }
→ Creates or updates invoice record for lab order

6. Lab Service · :3005

Purpose & Responsibility

Manages the laboratory: test catalog, patient registration for tests, result entry, and report packaging. Lab orders can be created in two ways:

1. From clinical session — doctor publishes lab.requested event → Lab Service auto-creates order
2. Walk-in — Lab Technician manually creates order for patients coming directly to lab

Database: lab schema — tests, orders, order_tests, results tables
File storage: MinIO hms-lab-reports/ for report PDFs

6.1 Test Catalog APIs

Method	Endpoint	Role(s)	Description
GET	/tests	any authenticated	Full list of available lab tests with prices.
POST	/tests	admin	Add a new test to the catalog.
PUT	/tests/:id	admin	Update test name or price.
DELETE	/tests/:id	admin	Remove test from catalog.

POST /tests — Request Body

Field	Type	Null	Notes
name	string	No	Test name e.g. 'Complete Blood Count'
code	string	No	Short code e.g. 'CBC'. Must be unique.
category	string	Yes	e.g. Haematology, Biochemistry
unit_price	decimal	No	Test cost
turnaround_hours	integer	Yes	Expected result turnaround time
reference_range	string	Yes	Normal range description

6.2 Lab Order APIs

Method	Endpoint	Role(s)	Description
POST	/orders	lab_tech	Register a patient for lab tests (walk-in). Compute total. Returns order_id.
GET	/orders	lab_tech, admin, doctor	List orders. Supports ?patient_id=&status=&date=
GET	/orders/:id	lab_tech, doctor, admin	Full order detail with tests and results.
PUT	/orders/:id/results	lab_tech	Enter test results for one or all tests in the order.

POST	/orders/:id/report	lab_tech	Upload packaged report PDF to MinIO. Marks order as completed. Returns report URL.
PUT	/orders/:id/pay	lab_tech	Record payment. Publishes billing.lab event to Finance.
GET	/patients/:id/history	lab_tech, doctor, admin	Full lab history for a patient — all orders and results.

POST /orders — Request Body

Field	Type	Null	Notes
patient_id	UUID	Yes	If null, patient_name and patient_mobile required
patient_name	string	Yes	Required if patient_id null
patient_mobile	string	Yes	Required if patient_id null
tests	array	No	Array of { test_id, urgency: routine|urgent }
notes	string	Yes	Notes for technician
source	enum	No	walk_in | clinical_request. Defaults to walk_in.
session_id	UUID	Yes	If source = clinical_request, link to originating session

PUT /orders/:id/results — Request Body

Field	Type	Null	Notes
results	array	No	Array of { order_test_id, result_value, is_abnormal: true/false, technician_notes }

POST /orders/:id/report — Request Body (multipart/form-data)

Field	Type	Null	Notes
report	file	No	PDF file. Max 20MB. Uploaded to MinIO hms-lab-reports/
notes	string	Yes	Lab notes for the doctor

6.3 RabbitMQ Event Consumer

Lab Service subscribes to lab.requested events from Clinical Service

lab.requested payload:
{ session_id, patient_id, patient_name, doctor_id, tests: [{ test_id, urgency }], notes }

Lab Service automatically:
1. Creates a lab order with source = clinical_request
2. Links to the session_id
3. Sets status = registered

> 4. Doctor can see the order status from GET /clinical/sessions/:id/lab-requests

7. Ward Service · :3006

Purpose & Responsibility

Manages the admission ward: bed availability, patient admissions, daily service stacking (adding charges for medication, procedures, meals etc.), and patient discharge with billing.

On discharge, publishes billing.ward event to Finance which closes the ward invoice.

Database: ward schema — wards, beds, admissions, ward_services tables
No file storage required for this service.

7.1 Ward & Bed APIs

Method	Endpoint	Role(s)	Description
GET	/wards	admin, ward_staff, doctor	List all wards with total beds, occupied count, and available count.
POST	/wards	admin	Create a new ward.
GET	/beds	admin, ward_staff	List beds. Supports ?ward_id=&status=available|occupied|maintenance
PUT	/beds/:id	admin	Update bed details or status (e.g. set to maintenance).

GET /wards — Response shape

```
[
  {
    "id": "uuid",
    "name": "Male General Ward",
    "type": "general",
    "capacity": 20,
    "occupied": 14,
    "available": 6,
    "maintenance": 0
  }
]
```

7.2 Admission APIs

Method	Endpoint	Role(s)	Description
POST	/admissions	ward_staff, admin	Admit a patient and assign a bed. Bed status changes to 'occupied'.
GET	/admissions	admin, ward_staff	List current admissions. Supports ?ward_id=&status=admitted|discharged

GET	/admissions/:id	admin, ward_staff, doctor	Full admission record.
PUT	/admissions/:id/discharge	ward_staff, admin	Discharge patient. Bed released. Triggers billing.ward event to Finance.
GET	/patients/:patient_id/admissions	admin, doctor, ward_staff	Full ward admission history for a patient.

POST /admissions — Request Body

Field	Type	Null	Notes
patient_id	UUID	No	
bed_id	UUID	No	Must be an available bed
attending_doctor_id	UUID	No	Responsible doctor
admission_reason	string	No	Reason for admission
notes	string	Yes	

PUT /admissions/:id/discharge — Request Body

Field	Type	Null	Notes
discharge_notes	string	Yes	Summary at discharge
discharge_diagnosis	string	Yes	Final diagnosis at time of discharge

PUT /admissions/:id/discharge — what happens internally

1. admission.status set to discharged, discharged_at set to now
2. bed.status set to available, bed.current_admission_id set to null
3. Fetch all ward_services for this admission to compute total
4. Publish billing.ward event:
   { admission_id, patient_id, patient_name, ward_name, bed_number, admitted_at, discharged_at, services: [...], total_amount }
5. Finance Service receives event and creates/closes ward invoice

7.3 Ward Service Stack APIs

The service stack is how charges accumulate during a patient's admission. Ward staff add entries for each service provided (medication, procedure, meal, investigation etc).

Method	Endpoint	Role(s)	Description
POST	/admissions/:id/services	ward_staff, doctor	Add a service/charge to the admission billing stack.

GET	/admissions/:id/services	ward_staff, admin	Get all services added to this admission with running total.
DELETE	/admissions/:id/services/:serviceId	ward_staff, admin	Remove a service entry (before discharge only).

POST /admissions/:id/services — Request Body

Field	Type	Null	Notes
service_name	string	No	Description e.g. 'IV Amoxicillin', 'Daily Bed Charge', 'Blood Transfusion'
service_type	enum	No	medication | procedure | bed_charge | investigation | meal | other
quantity	integer	No	
unit_price	decimal	No	
notes	string	Yes	
provided_at	datetime	Yes	Defaults to now

8. Database Schema Reference

Single PostgreSQL instance with 6 schemas — one per service. Services never query another service's schema. Cross-service references use UUID foreign keys stored as TEXT (no DB-level FK constraints across schemas).

8.1 reception schema

Field	Type	Null	Notes
patients.id	UUID	No	Primary key
patients.patient_number	VARCHAR(20)	No	Unique e.g. R00001
patients.first_name	VARCHAR(100)	No	
patients.last_name	VARCHAR(100)	No	
patients.mobile	VARCHAR(20)	No	Unique — used for search
patients.email	VARCHAR(200)	Yes	
patients.date_of_birth	DATE	No	
patients.gender	VARCHAR(10)	No	male|female|other
patients.address	TEXT	Yes	
patients.blood_group	VARCHAR(5)	Yes	
patients.avatar_url	TEXT	Yes	MinIO object key
patients.triage	VARCHAR(20)	Yes	non_urgent|emergency|out_patient|urgent
patients.status	VARCHAR(20)	No	active|inactive
patients.created_at	TIMESTAMPTZ	No	
tokens.id	UUID	No	
tokens.patient_id	UUID	No	Ref: patients.id
tokens.token_number	SMALLINT	No	Sequential per day per queue
tokens.queue_type	VARCHAR(30)	No	opd|wound_care|doctor_consultation
tokens.doctor_id	UUID	Yes	Ref: staff.doctors.id
tokens.status	VARCHAR(20)	No	waiting|called|in_progress|completed|skipped|cancelled
tokens.session_date	DATE	No	Token resets daily
tokens.issued_by	UUID	No	Receptionist user ID
appointments.id	UUID	No	
appointments.patient_id	UUID	Yes	
appointments.doctor_id	UUID	No	
appointments.appointment_date	DATE	No	

appointments.from_time	TIME	No	
appointments.to_time	TIME	No	
appointments.treatment	VARCHAR(200)	Yes	
appointments.status	VARCHAR(30)	No	scheduled|ongoing|completed|cancelled|rescheduled
appointments.booked_by	UUID	No	Receptionist user ID

8.2 clinical schema

Field	Type	Null	Notes
sessions.id	UUID	No	
sessions.patient_id	UUID	No	Ref: reception.patients.id
sessions.doctor_id	UUID	No	Ref: staff.doctors.id
sessions.nurse_id	UUID	Yes	Ref: staff.staff.id
sessions.session_type	VARCHAR(20)	No	opd|wound_care|channelling
sessions.appointment_id	UUID	Yes	Ref: reception.appointments.id
sessions.chief_complaint	TEXT	Yes	
sessions.diagnosis	TEXT	Yes	
sessions.follow_up_date	DATE	Yes	
sessions.status	VARCHAR(20)	No	open|completed
sessions.discount_percent	DECIMAL(5,2)	Yes	
sessions.discount_reason	TEXT	Yes	
sessions.completed_at	TIMESTAMPTZ	Yes	
sessions.created_at	TIMESTAMPTZ	No	
vitals.id	UUID	No	
vitals.session_id	UUID	No	
vitals.bpm	INTEGER	Yes	
vitals.temperature	DECIMAL(4,1)	Yes	Celsius
vitals.blood_pressure_sys	INTEGER	Yes	
vitals.blood_pressure_dia	INTEGER	Yes	
vitals.spo2	INTEGER	Yes	
vitals.weight_kg	DECIMAL(5,2)	Yes	
vitals.height_cm	INTEGER	Yes	
vitals.blood_sugar	DECIMAL(5,2)	Yes	

prescriptions.id	UUID	No	
prescriptions.session_id	UUID	No	
prescriptions.type	VARCHAR(20)	No	internal|external
prescriptions.medicine_name	VARCHAR(200)	No	
prescriptions.dosage	VARCHAR(100)	No	
prescriptions.frequency	VARCHAR(100)	No	
prescriptions.duration_days	INTEGER	No	
prescriptions.status	VARCHAR(20)	No	pending|dispensed (internal only)
session_images.id	UUID	No	
session_images.session_id	UUID	No	
session_images.file_url	TEXT	No	MinIO object key
session_images.caption	TEXT	Yes	
session_images.image_type	VARCHAR(20)	No	scan|wound|other

8.3 finance schema

Field	Type	Null	Notes
invoices.id	UUID	No	
invoices.invoice_number	VARCHAR(30)	No	INV-2026-00001, unique
invoices.patient_id	UUID	No	
invoices.patient_name	VARCHAR(200)	No	Denormalized for audit integrity
invoices.billing_module	VARCHAR(30)	No	opd|wound_care|channelling|lab|ward|pharmacy
invoices.session_reference_id	UUID	Yes	
invoices.subtotal	DECIMAL(12,2)	No	
invoices.discount_amount	DECIMAL(12,2)	No	Defaults 0
invoices.total_amount	DECIMAL(12,2)	No	subtotal - discount_amount
invoices.payment_status	VARCHAR(20)	No	pending|paid|partial|waived
invoices.payment_method	VARCHAR(20)	Yes	cash|card|online|insurance
invoices.paid_at	TIMESTAMPTZ	Yes	
invoices.source	VARCHAR(20)	No	event|manual
invoices.created_at	TIMESTAMPTZ	No	
line_items.id	UUID	No	
line_items.invoice_id	UUID	No	

line_items.item_name	VARCHAR(200)	No	
line_items.item_type	VARCHAR(50)	No	consultation|procedure|medicine|lab_test|bed_charge|other
line_items.quantity	SMALLINT	No	
line_items.unit_price	DECIMAL(10,2)	No	
line_items.total_price	DECIMAL(10,2)	No	

8.4 lab schema

Field	Type	Null	Notes
tests.id	UUID	No	
tests.name	VARCHAR(200)	No	
tests.code	VARCHAR(20)	No	Unique
tests.category	VARCHAR(100)	Yes	
tests.unit_price	DECIMAL(10,2)	No	
tests.turnaround_hours	INTEGER	Yes	
orders.id	UUID	No	
orders.patient_id	UUID	Yes	
orders.patient_name	VARCHAR(200)	No	Denormalized
orders.session_id	UUID	Yes	From clinical_request
orders.source	VARCHAR(20)	No	walk_in|clinical_request
orders.status	VARCHAR(30)	No	registered|processing|completed|cancelled
orders.total_amount	DECIMAL(10,2)	No	
orders.payment_status	VARCHAR(20)	No	pending|paid
orders.report_file_url	TEXT	Yes	MinIO key
orders.created_at	TIMESTAMPTZ	No	
order_tests.id	UUID	No	
order_tests.order_id	UUID	No	
order_tests.test_id	UUID	No	
order_tests.test_name	VARCHAR(200)	No	Denormalized
order_tests.unit_price	DECIMAL(10,2)	No	
order_tests.result_value	TEXT	Yes	
order_tests.is_abnormal	BOOLEAN	Yes	
order_tests.status	VARCHAR(20)	No	pending|completed

8.5 ward schema

Field	Type	Null	Notes
wards.id	UUID	No	
wards.name	VARCHAR(100)	No	
wards.type	VARCHAR(30)	No	general|icu|private|semi_private
wards.capacity	SMALLINT	No	
beds.id	UUID	No	
beds.ward_id	UUID	No	
beds.bed_number	VARCHAR(20)	No	e.g. B-101
beds.status	VARCHAR(20)	No	available|occupied|maintenance|reserved
beds.current_admission_id	UUID	Yes	
admissions.id	UUID	No	
admissions.patient_id	UUID	No	
admissions.patient_name	VARCHAR(200)	No	
admissions.bed_id	UUID	No	
admissions.ward_id	UUID	No	
admissions.attending_doctor_id	UUID	No	
admissions.admission_reason	TEXT	No	
admissions.admitted_at	TIMESTAMPTZ	No	
admissions.discharged_at	TIMESTAMPTZ	Yes	
admissions.status	VARCHAR(20)	No	admitted|discharged
ward_services.id	UUID	No	
ward_services.admission_id	UUID	No	
ward_services.service_name	VARCHAR(200)	No	
ward_services.service_type	VARCHAR(50)	No	
ward_services.quantity	SMALLINT	No	
ward_services.unit_price	DECIMAL(10,2)	No	
ward_services.total_price	DECIMAL(10,2)	No	
ward_services.provided_at	TIMESTAMPTZ	No	

9. 3-Day Implementation Plan

Quick start stack — set up in 30 minutes with Docker Compose

```
docker-compose.yml services:
  postgres:15 — one instance, create 6 schemas in init.sql
  minio/minio — object storage
  rabbitmq:3-management — message broker (management UI on :15672)
  jboss/keycloak — auth server

Each microservice:
  npm init + npm install express pg @aws-sdk/client-s3 amqplib jsonwebtoken dotenv multer
  One index.js, one routes folder, one db.js (pg pool), one minio.js, one rabbitmq.js
```

Day 1 — Foundation + Reception + Staff

- Morning: Set up Docker Compose with Postgres, MinIO, RabbitMQ, Keycloak
- Configure Keycloak realm hms, create roles, create test users
- Run Postgres init.sql to create all 6 schemas and tables
- Scaffold all 6 service folders with Express skeleton, JWT middleware, pg pool
- Afternoon: Build Reception Service — patients CRUD, token issuance, queue GET, appointment CRUD
- Build Staff Service — doctors CRUD + Keycloak Admin API calls, departments, schedules, nurse allocations
- Test with Postman — patient registration, login, token issued, queue shows up

Day 2 — Clinical + Lab + Finance

- Morning: Build Clinical Service — sessions, vitals, diagnoses, images (MinIO), prescriptions
- Wire RabbitMQ publisher: on session complete → publish billing.opd
- Wire RabbitMQ publisher: on lab request → publish lab.requested
- Afternoon: Build Lab Service — test catalog, orders, results, report upload (MinIO)
- Wire Lab Service RabbitMQ consumer: lab.requested → auto-create order
- Build Finance Service — invoices, line items, payments, discount, reports
- Wire Finance RabbitMQ consumer: billing.* → auto-create invoices
- Test end-to-end: patient visit → session complete → invoice auto-created

Day 3 — Ward + API Gateway + Polish

- Morning: Build Ward Service — beds, admissions, service stack, discharge
- Wire discharge → billing.ward event → Finance closes invoice
- Build API Gateway — JWT validation middleware, route proxying, WebSocket bridge for queue display
- Afternoon: Wire WebSocket + RabbitMQ queue.events fanout for Queue Display board
- Build dashboard aggregation endpoints (GET /reports/summary, /reports/earnings etc.)
- Docker Compose integration test of full system
- Fix bugs, clean up error handling, write README

9.1 Keycloak Setup Checklist

- Create realm: hms
- Create client: hms-app (public client, direct access grants enabled)
- Create roles: admin, doctor, nurse, receptionist, lab_tech, ward_staff
- Create service account for Staff Service to call Admin API (confidential client)
- Create test user for each role for Postman testing
- Export realm config as hms-realm.json for Docker init mount

9.2 Environment Variables (per service .env)

```
PORT=3001
DATABASE_URL=postgresql://postgres:password@postgres:5432/hms
DB_SCHEMA=reception
KEYCLOAK_URL=http://keycloak:8080
KEYCLOAK_REALM=hms
RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672
MINIO_ENDPOINT=minio
MINIO_PORT=9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET=hms-scans
```